

AULA 12: Implementação da Unidade de Controle

Samuel Tomazela e Eric Vicente

1) Apresente o código fonte, em Verilog, do módulo que implementa a Unidade de Controle do seu nRisc. [4 pontos]

```
module unidade_controle(
    input [3:0] opCode,
    output reg branch,    // (Beqz) desvio condicional
    output reg memRead,   // leitura da mem
    output reg memToReg,
    output reg memWrite,  // EscMEM - escrita na mem
    output reg regWrite,  // EscReg - escrita no reg
    output reg ALUSrc,
    output reg ALUOp,     // 0-add 1-sub
    output reg JiR,       // JMPR salto relativo
    output reg Ji,        // JMP salto absoluto
    output reg EscPC,     // HALT
    output reg [1:0] sel_MemToReg, // seletor do mux para salvar dados
    output reg [1:0] sel_ALUSrc  // seletor mux ula
);

always @(*) begin
    // Valores default (Evita inferred latches)
    branch    = 0;
    memRead   = 0;
    memWrite  = 0;
    regWrite  = 0;
    ALUOp     = 0;
    JiR       = 0;
    Ji        = 0;
    EscPC     = 1;
    sel_MemToReg = 2'b00;
    sel_ALUSrc  = 2'b00;

    case(opCode)
        4'b0000: begin // ADD
            regWrite = 1;
        end
        4'b0001: begin // LI
            regWrite  = 1;
            sel_ALUSrc = 2'b01;
            sel_MemToReg = 2'b10;
        end
        4'b0010: begin // LW
            regWrite  = 1;

```

```

        memRead    = 1;
        sel_MemToReg = 2'b01;
    end
    4'b0011: begin // SW
        memWrite = 1;
    end
    4'b0100: begin // BEQ
        sel_ALUSrc = 2'b10; // R0 fixo
        ALUOp      = 1;
        branch     = 1;
    end
    4'b0101: begin // JMP (Absoluto)
        Ji = 1;
    end
    4'b0110: begin // SUB
        regWrite = 1;
        ALUOp    = 1;
    end
    4'b0111: begin // JMPR (Relativo)
        JiR = 1;
    end
    4'b1111: begin // HALT
        EscPC = 0;
    end
    default: EscPC = 1;
endcase

ALUSrc = (sel_ALUSrc != 2'b00);
memToReg = (sel_MemToReg != 2'b00);
end

endmodule

```

2) Configure simulações que demonstrem o correto funcionamento do componente implementado. Apresente o código fonte desse módulo de simulação. [2 pontos]

```

module tb_unidade_controle;
    reg [3:0] opCode;

    // Sinais de saída da UC
    wire branch, memRead, memToReg, memWrite, regWrite, ALUSrc, ALUOp, JiR, Ji,
    EscPC;
    wire [1:0] sel_MemToReg, sel_ALUSrc;

    // Instanciação do módulo unidade_controle
    unidade_controle uut (
        .opCode(opCode),

```

```

        .branch(branch),
        .memRead(memRead),
        .memToReg(memToReg),
        .memWrite(memWrite),
        .regWrite(regWrite),
        .ALUSrc(ALUSrc),
        .ALUOp(ALUOp),
        .JiR(JiR),
        .Ji(Ji),
        .EscPC(EscPC),
        .sel_MemToReg(sel_MemToReg),
        .sel_ALUSrc(sel_ALUSrc)
    );

```

```

initial begin

```

```

    // Cabeçalho do monitor textual
    $display("OpCode | branch | memRead | memWrite | regWrite | ALUOp | EscPC |
sel_MTR | sel_AS");
    $display("-----");

    // 1. Teste ADD (0000)
    opCode = 4'b0000; #10;
    $display(" 0000 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

    // 2. Teste LI (0001)
    opCode = 4'b0001; #10;
    $display(" 0001 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

    // 3. Teste LW (0010)
    opCode = 4'b0010; #10;
    $display(" 0010 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

    // 4. Teste SW (0011)
    opCode = 4'b0011; #10;
    $display(" 0011 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

    // 5. Teste BEQ (0100)
    opCode = 4'b0100; #10;
    $display(" 0100 | %b | %b | %b | %b | %b | %b | %b | %b",

```

```

        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

// 6. Teste JMP (0101)
opCode = 4'b0101; #10;
$display(" 0101 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

// 7. Teste SUB (0110)
opCode = 4'b0110; #10;
$display(" 0110 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

// 8. Teste JMPR (0111)
opCode = 4'b0111; #10;
$display(" 0111 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

// 9. Teste HALT (1111)
opCode = 4'b1111; #10;
$display(" 1111 | %b | %b | %b | %b | %b | %b | %b | %b",
        branch, memRead, memWrite, regWrite, ALUOp, EscPC, sel_MemToReg,
sel_ALUSrc);

$display("-----");
$display("Fim da Simulacao");
$finish;
end
endmodule

```

3) Demonstre e explique o funcionamento das simulações, usando fotos da tela (screenshots) da aplicação ModelSim em execução. [2 pontos]

